

SECTION 28 — DEBUGGING AND FAILURE ANALYSIS (Refined)

Topics Covered

- 28.1-28.18 (18 headers)

First pass — 2 parallel batch subagents

28.1 Test Fails Locally and in CI — Refined

Theory: Fails everywhere = deterministic bug, not flake. Reproduce with `npx playwright test file:line --headed --debug`; open HTML report + trace viewer (actions, console, network, screenshots). Check `git blame` for recent app/locator/data/API changes. Replace hard sleeps with web-first assertions + auto-waiting. Rerun from clean state: fresh context, cleared `storageState`, reseeded database. Fix test or app code, add regression assertion, verify locally across Chromium/Firefox/WebKit. Enterprise Relevance: Deterministic failures are the easiest class — reproduce, fix, guard. Every such failure must produce a regression test, or it will recur.

```
npx playwright test checkout.spec.ts:42 --headed --debug
npx playwright show-trace test-results/trace.zip # actions + console + network
timeline
```

High-Stakes Scenario: Same failure 2 weeks, "known issue" — no owner, no ticket. Triage: deterministic = must-fix (not quarantine), git-blame to owning change, regression test attached, cross-browser verification before close. Anti-Patterns: Calling deterministic failures "flaky"; no regression test after fix; debugging without trace (guessing); verifying only one browser. Qs: How do you prove deterministic vs flaky? What must every fix include?

28.2 Test Fails Only in CI — Refined

Theory: Passes locally, fails in CI = environment parity issue, not logic. Compare dimensions: headless vs headed, viewport, timezone/locale, fonts, CPU throttling (CI slower), `npx playwright install --with-deps` / Docker image versions, `baseURL` / `webServer` / `secrets/service` URLs in workflow YAML. Enable `trace/screenshot/video on-first-retry`, upload `playwright-report` artifacts. Slower CI needs longer `expect` timeouts, proper waits (not sleeps), blocked-network/CORS checks. Stabilize, then rerun sharded workflow to confirm green. Enterprise Relevance: "Works on my machine" is a parity bug. CI/local parity checklist (browser version, viewport, tz, deps, services, secrets) resolves 80% of CI-only failures without code changes.

```
# parity essentials in workflow:
- run: npx playwright install --with-deps chromium
- run: npx playwright test --reporter=blob
  env: { TZ: UTC, BASE_URL: ${ secrets.QA_URL } }
```

High-Stakes Scenario: Timezone-dependent test (midnight UTC rollover) passes in IST local, fails in UTC CI at 00:30. Triage: pin `TZ=UTC` everywhere, freeze clocks for date logic, locale/fonts parity, CPU-throttle reproduction locally (`--cpu-throttling`). Anti-Patterns: Raising timeouts instead of finding

parity gap; debugging without CI artifacts; "CI is flaky" dismissal; local-only env files. Qs: List the CI-vs-local parity dimensions? How do traces replace local repro?

28.3 Test Fails Only in Parallel — Refined

Theory: Passes serially, fails parallel = shared mutable state / isolation failure. Check: `workers`, `fullyParallel`, `describe.serial`, sharding, worker- vs test-scoped fixtures. Look for collisions: test users, DB rows, files, ports, storage files, sessions, global setup. Give each worker unique data (`workerIndex`, parallel-safe APIs), no singleton pages. Confirm with `--workers=1` vs `--workers=4`. Enforce independent contexts, seeded data cleanup, no cross-test dependencies. Enterprise Relevance: Parallel-only failures are isolation bugs (Sec 22-23) surfacing as "flakes". The `--workers=1` vs N differential is the definitive diagnostic.

```
npx playwright test --workers=1 # green → isolation bug, not product
npx playwright test --workers=4 # red → find shared resource via collision grouping
```

High-Stakes Scenario: Parallel rollout: 35% failures, all duplicate-key/port-collision. Triage: collision-group failures by error (same constraint = same shared entity), per-worker namespacing, `fullyParallel` CI job as merge gate for new tests. Anti-Patterns: Serializing suite to "fix" (hides bug, loses speed); retries masking collisions; blaming "parallel instability". Qs: What does `workers=1` vs 4 prove? List 5 shared resources to check first?

28.4 Intermittent Failure — Refined

Theory: Intermittent = flaky until proven otherwise. Run `--repeat-each=20 --workers=1 + --retries=2`; separate `passed` / `flaky` (failed-then-passed) / `failed` in HTML report. Inspect traces across runs for races, animations, lazy loading, network jitter, timers, ordering assumptions. Replace fixed sleeps with `expect(locator).toBeVisible()`, `toHaveText()`, `waitForResponse()`; stabilize mocks with `page.route()`; control time with `clock`. Use `testInfo.retry` for cleanup; quarantine flaky test; fix root cause; remove retry crutch. Enterprise Relevance: Intermittent failures are triaged by pattern, not anecdote: 1/20 = timing race; 10/20 = order/data dependence; 20/20 = deterministic (go to 28.1). Repeat runs quantify; traces explain.

```
npx playwright test flaky.spec.ts --repeat-each=20 --workers=1 --retries=0
# 18 pass / 2 fail → timing race; 0 pass alone-fail / suite-fail → order dependence
```

High-Stakes Scenario: "Flaky, rerun" culture — 15% suite intermittent, no quantification, real bugs hide in noise. Triage: flake-rate per test (repeat runs), quarantine >5%, root-cause categories (wait/data/order/env), fix in priority order, retry budget (2 CI, 0 local). Anti-Patterns: Single rerun as "proof"; retries without classification; `waitForTimeout` fixes; no quarantine (noise blocks all). Qs: How do you quantify intermittence? What distinguishes timing race from order dependence?

28.5 Timeout Failure — Refined

Theory: Identify which timeout fired: test (30s), `expect` (5s), `actionTimeout`, `navigationTimeout`, fixture, or `globalTimeout`. Read the call log in trace: waited selector, assertion, navigation, or API call. Causes: slow rendering, missing `await` (promise never resolved), hung backend, large payload, CI resource starvation. Prefer fixing waits and performance over raising limits; use `test.slow()`, `test.setTimeout()`, per-assertion `{ timeout }` deliberately. Ensure `webServer` readiness, API health, `waitForLoadState`; verify without excessive durations. Enterprise Relevance: Timeout tuning

without diagnosis masks real slowness (a 25s page load "fixed" by 60s timeout ships to users). Every timeout increase needs a linked performance ticket.

```
// deliberate, documented timeout (not blind raise):
await expect(page.getByText('Export complete'), 'export job slow by
design').toBeVisible({ timeout: 30_000 });
test.slow(); // triples budgets for known-slow suite
```

High-Stakes Scenario: Missing `await` on click → action never completes → 30s timeout, misdiagnosed as "slow app" for weeks. Triage: call log shows action never started (vs started-but-slow); no-floating-promises lint prevents the class; trace actionability timeline distinguishes. Anti-Patterns: Global timeout raises; `test.setTimeout(300000)` blanket; no call-log reading; timeout as "stability fix". Qs: Which timeout fired — how do you tell? When is raising a timeout correct vs masking?

28.6 Locator Failure — Refined

Theory: Treat as selector ambiguity or timing issue. Open trace viewer + Playwright Inspector (`codegen` , `locator()`) to validate `getByRole` / `getByTestId` / `getByLabel` / `getByText` resolution. Check: strict-mode violations (multiple matches), hidden elements, iframes, shadow DOM, dynamic IDs, localization, viewport changes. Scope with `filter()` , `nth()` , `hasText` , test IDs; assert visibility before action. Avoid layout-tied XPath/CSS. Update Page Object; rerun headed; confirm stable resolution across browsers. Enterprise Relevance: Locator failures are the most common UI failure class — systematic validation (strict + visibility + cross-browser) replaces guesswork. Every locator fix updates the POM, never the test.

```
npx playwright codegen https://app.com # validate resolution live
# Inspector: locator count must show 1/1 (strict), visible, stable
```

High-Stakes Scenario: Design-system upgrade renames 200 classes — 150 locator failures, one root cause. Triage: bulk-migrate to role/testid locators (not 150 XPath patches), locator-strategy review, visual diff to catch silent layout changes. Anti-Patterns: XPath patching per failure (whack-a-mole); `nth()` to silence strict errors; layout-tied selectors; test-local locators (not POM). Qs: Strict violation vs timing vs wrong frame — how do you distinguish? What locator would survive a redesign?

28.7 Stale Element — Refined

Theory: In Playwright this usually means detached DOM or bad `ElementHandle` use (stored handle across rerender). Prefer resilient `Locator` (re-queries automatically) over stored handles or `eval` references across renders. Causes: React re-render, list virtualization, navigation, DOM replacement between find and click. Add web-first assertion (`toBeAttached()` , `toBeVisible()` , `waitFor()`) before interaction. Re-locate inside retry loop; never cache elements across steps. Verify with trace filmstrip; refactor helper to accept locator factory, not stale node. Enterprise Relevance: Stale-element class disappears with Locator discipline — its presence in a suite signals Handle-era patterns or cached elements needing refactor.

```
// BAD: cached handle across rerender
const h = await page.$('#price'); await rerender(); await h.click(); // stale
// GOOD: locator re-resolves
```

```
const price = page.getByTestId('price');
await expect(price).toBeAttached(); await price.click();
```

High-Stakes Scenario: Helper accepts `ElementHandle` param — every caller caches, stale everywhere after React upgrade. Triage: refactor helpers to `Locator | () => Locator` factory params; lint ban on `page.$` in new code; `toBeAttached` guards before actions on dynamic lists. Anti-Patterns: `page.$` + stored handles; caching locators' resolved nodes; `eval` references across steps; retry loops reusing the same handle. Qs: `Locator` vs `ElementHandle` — why does one go stale? How do you refactor Handle-based helpers?

28.8 Authentication Failure — Refined

Theory: Isolate session, token, and setup flow first. Inspect: global setup, `storageState` path, project dependencies, session expiry, refresh rotation, MFA, SSO redirects, clock skew. Check: missing CI secrets, wrong environment credentials, parallel workers overwriting shared auth file, invalid test user. Re-run setup with `--project=setup --debug`; validate cookies and `localStorage`; use unique users per worker. Distinguish 401 (who-are-you) vs 403 (what-may-you) vs redirect loops (SSO misconfig). Enterprise Relevance: Auth failures cascade — one expired setup token fails 200 tests with misleading errors. Setup-project health is a suite-level dashboard item, not per-test noise.

```
npx playwright test --project=setup --debug # isolate setup from suite
# check: storageState file fresh? cookies valid? per-worker files distinct?
```

High-Stakes Scenario: Shared `auth.json` overwritten by parallel workers (each login replaces it) — random 401s across suite. Triage: per-worker auth files (`auth-w${workerIndex}.json`), setup sharding by role, token-expiry buffer (refresh at 80% lifetime), secret rotation calendar. Anti-Patterns: Shared auth file parallel; committed auth state; no setup-project isolation (auth failure looks like 200 test failures); ignoring 401-vs-403 distinction. Qs: 401 vs 403 vs redirect loop — what layer failed? How do you isolate setup failures from test failures?

28.9 Environment Failure — Refined

Theory: Suspect configuration and dependencies before product code. Verify: `baseUrl`, `.env`, `webServer` command and readiness, database seed and migrations, feature flags, third-party services, network allowlists. Compare Node, browser, OS, Docker image, Playwright versions local vs CI. Check: ports in use, disk space, permissions, proxy, certificates, flaky external APIs (mock where appropriate). Capture logs, health checks, artifact reports. Pin versions, document required services, add startup readiness checks, rerun full pipeline. Enterprise Relevance: Environment failures are suite-external — fixing tests for env problems creates test debt. Readiness checks + version parity + health endpoints distinguish env from product in minutes.

```
# readiness gate before suite (fail fast on env, not 200 confusing test failures):
curl -f $BASE_URL/health && npx prisma migrate status && node check-services.js
```

High-Stakes Scenario: Staging DB migration not applied — 300 tests fail on missing column, triaged as "product regression" for a day. Triage: migration-status check in setup (fail fast with clear message), version parity dashboard (app vs test vs infra versions), service health pre-flight. Anti-Patterns: Testing against unready env (waste); version drift local-vs-CI; no readiness checks; mocking nothing (third-party flake blocks all). Qs: Env vs product vs test failure — decision tree? What readiness checks gate your suite?

28.10 Network Failure — Refined

Theory: Network failures present as `ECONNREFUSED`, `ECONNRESET`, `TimeoutError`, aborted `page.goto`, or failed API responses. Triage: 1) Check trace `requests --failed` + console errors. 2) Verify `baseURL`, `webServer` status, ports, `localhost` vs `127.0.0.1` IPv6 mismatch. 3) Reproduce with `context.setOffline(false)` and inspect HAR. 4) Distinguish app timeout handling (via `page.route abort/delay` reproduction) from infra outage before retrying. Enterprise Relevance: Network failures split into app behavior (timeouts, retries, offline UX — testable) vs infra outage (not testable — quarantine). Reproducing with route faults proves which.

```
// prove app handles timeout gracefully (vs infra flake):
await page.route('**/api/orders', r => r.abort('timedout'));
await expect(page.getByTestId('error')).toHaveText(/try again/);
```

High-Stakes Scenario: IPv6 `localhost` resolves `::1` but server listens IPv4 only — CI-only `ECONNREFUSED`, "flaky network" for months. Triage: explicit `127.0.0.1` vs `localhost` test, `webServer host: 0.0.0.0` binding check, HAR comparison local-vs-CI. Anti-Patterns: Retrying infra outages as test flakes; no HAR/trace capture; `localhost` assumptions across OSs; testing offline UX never (missing feature). Qs: App timeout vs infra outage — how do you distinguish? What does HAR add over error text?

28.11 Database Failure — Refined

Theory: Database failures show as connection refused, migration mismatch, empty query results, or seed timeouts. Triage: 1) Check connection string, container health (`docker ps`, `pg_isready`), migration version (`prisma migrate status`). 2) Query row counts for expected seed users/orders. 3) Inspect test logs for constraint violations or truncated tables. 4) Verify isolation: shared DB, parallel workers overwriting rows, missing per-test reseed. Fix environment and reseeding in `beforeEach`, not by masking with retries. Enterprise Relevance: DB failures are env/data, never product — but they produce hundreds of misleading test failures. Migration-gated setup (fail fast with version message) converts 300 confusing failures into 1 clear one.

```
npx prisma migrate status # pending migrations? fail fast here
SELECT COUNT(*) FROM users WHERE run_id = 'run412'; -- seed present?
```

High-Stakes Scenario: Migration applied to CI DB but seed job skipped — all tests fail "user not found", triaged as product regression for a day. Triage: setup asserts seed counts (fail fast: "seed missing: 0 users, expected 50"), migration+seed version logged per run, reseed in `beforeEach` for test-scoped data. Anti-Patterns: Retrying DB-connection failures (env won't heal); shared DB without run scoping; no migration check; truncation without scoping (Sec 23.7). Qs: Connection vs migration vs seed vs isolation failure — how do you distinguish fast?

28.12 Dependency Failure — Refined

Theory: Dependency failures occur when third-party APIs, auth providers, payment gateways, or mocks return 4xx/5xx, timeouts, or rate limits. Triage: 1) Identify external call in trace network log. 2) Check status, `Retry-After`, contract drift, sandbox outage page. 3) Reproduce against `WireMock/MockServer` stub or recorded fixture. 4) Verify `page.route` stubs match method, URL, and headers (stub mismatch mimics dependency failure). Quarantine as environment issue; add contract

test + controlled negative stubs (dependency-down behavior). Enterprise Relevance: Third-party failures are the most common "not our bug" that still blocks releases. Stub-by-default (recorded fixtures) + one live contract test contains the blast radius.

```
// stub mismatch check: method + URL + headers must all match
await page.route('**/api/pay', r => r.fulfill({ status: 200, body: JSON.stringify({
id: 'pay1' }) }));
// vs live contract test (nightly, quarantined on sandbox outage):
```

High-Stakes Scenario: Payment sandbox deploys breaking change Friday — 100 E2E fail all weekend, Monday release blocked. Triage: stub third-party in PR suites (recorded fixtures), live contract test nightly (catches drift), sandbox status check in setup (fail fast with "sandbox down", not 100 confusing failures). Anti-Patterns: Live third-party in PR suites; stub mismatch (wrong method/URL) misdiagnosed as dependency outage; no contract test (drift undetected until E2E breaks). Qs: Stub vs live contract test — what does each prove? How do you tell stub-mismatch from real outage?

28.13 Test Data Failure — Refined

Theory: Test-data failures show as "expected 3, got 0", missing IDs, stale schemas, or order-dependent passes. Triage: 1) Confirm seed source, schema version, and hardcoded IDs (like user 42 — fragile). 2) Check parallel leakage, leftover rows, cross-test pollution. 3) Rerun isolated with fresh deterministic seed and unique emails. 4) Inspect fixtures for drift after model changes. Fix with schema-driven seeds per run, explicit setup, transactional rollback (Sec 23). Enterprise Relevance: Data failures are the silent majority of "flakes" — deterministic given the data state, random across runs. Data-first triage (what rows exist?) before code triage saves hours.

```
-- data-first triage queries:
SELECT COUNT(*) FROM users WHERE run_id = 'run412'; -- seed present?
SELECT * FROM users WHERE email = 'hardcoded@test.com'; -- collision/leftover?
```

High-Stakes Scenario: Hardcoded user 42 deleted by another test's cleanup — 20 tests fail "user not found", pass alone. Triage: owned-IDs rule (tests use IDs they created), hardcoded-ID lint, isolated rerun confirms data (not code), fixture drift check after model changes. Anti-Patterns: Hardcoded IDs; "latest row" queries; shared seed files mutated per test; no data-first triage (debugging code for hours when rows are missing). Qs: Data vs code vs env failure — what query answers first? Why are hardcoded IDs fragile?

28.14 Browser Compatibility Failure — Refined

Theory: Browser-compatibility failures pass in Chromium but fail in Firefox/WebKit due to viewport, focus, protocol, or rendering differences. Triage: 1) Run same spec per project (--project=firefox/webkit) to isolate engine. 2) Compare screenshots, DOM snapshots, launch errors. 3) Check Playwright/browser version match and devices viewport. 4) Replace Chromium-only selectors and timing assumptions with auto-waiting locators before filing engine-specific bug (most "engine bugs" are test assumptions). Enterprise Relevance: Cross-browser matrix exists because clients use all engines. Per-project runs in CI + engine-specific quarantine (not suite-wide ignore) keep matrix green honestly.

```
npx playwright test checkout.spec.ts --project=chromium # green
```

```
npx playwright test checkout.spec.ts --project=webkit # red → engine-specific
```

High-Stakes Scenario: Date input works Chromium, fails WebKit (native picker differences) — 30 failures, all date fields. Triage: fill via `fill()` (not key-by-key typing), locale-aware date formats, per-engine baseline screenshots, engine-specific skip with ticket (not silent). Anti-Patterns: Chromium-only development (matrix always red); engine-specific skips without tickets; pixel-diff across engines (font rendering differs); assuming CDP APIs on Firefox. Qs: Engine bug vs test assumption — how do you prove which? What belongs in per-engine baselines?

28.15 Collecting Diagnostic Artifacts — Refined

Theory: Diagnostic artifacts prove failure context for CI-only flakes (no local repro possible). Configure: `trace: on-first-retry`, `screenshot: only-on-failure`, `video: retain-on-failure`. Upload `test-results/` + `trace.zip` via `actions/upload-artifact`. Replay with `npx playwright show-trace` (or `trace actions --errors-only`), snapshots, requests, console. Attach logs, HTML, `testInfo.retry` metadata to report. Treat traces as sensitive (may contain secrets/PII — restrict access, redact). Enterprise Relevance: Artifacts are the difference between "CI failed, rerun?" and "CI failed, trace shows API 500 on payment call at 10:02, ticket filed". Every CI failure without artifacts is untriageable by definition.

```
// playwright.config.ts — evidence balance (insight vs storage):
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' }
```

```
- uses: actions/upload-artifact@v4
  if: failure()
  with: { name: failure-evidence, path: test-results/, retention-days: 14 }
```

High-Stakes Scenario: Flaky CI failure with zero artifacts — 3 engineers spend a day trying to repro locally (never reproduces). Triage: artifacts-first policy (no artifact = process failure, fix config before test), trace timeline reading skill (actions → console → network order), secrets redaction verified. Anti-Patterns: No artifacts in CI; `trace: on` always (storage/cost); artifacts expired before triage (1-day retention); traces with secrets in public artifacts. Qs: Trace vs screenshot vs video — what does each answer? How do you handle secrets in artifacts?

28.16 Root Cause vs Symptom — Refined

Theory: Symptoms are timeout/assert/crash messages; root causes are async waits, shared state, network issues, or data drift underneath. Triage: 1) Capture exact error, stack, line, DOM before/after. 2) Check co-occurring failures + history (systemic flakiness pattern?). 3) Bisect: isolate test, stub network, reseed data — which change fixes it identifies the layer. 4) Classify as flaky/regression/environment before fixing. Fix test isolation/waiting or product code — never just the error text (raising timeout for a race). Enterprise Relevance: Symptom-fixing ("increase timeout", "add retry") without root-cause classification is how suites accumulate 30% flake. Every failure gets a layer label before a fix.

```
Symptom: Timeout 30s on .results
Bisect: isolated+reseeded+stubbed → passes → data layer; still fails → wait/app layer
```



```
Cause classes: async-wait / shared-state / network / data-drift / product-regression
Fix must match class (wait fix for wait cause – never timeout raise for race).
```

High-Stakes Scenario: Team "fixes" 50 timeouts by raising timeouts 5s→30s — suite 3x slower, flakes persist (races, not slowness). Triage: timeout raises require slowness evidence (call log shows action started but slow); race evidence (action never started / DOM changed) requires waits/isolation instead. Anti-Patterns: Timeout raises for races; retries for deterministic failures; fixing error text (matchers) instead of cause; no classification (all failures "flaky"). Qs: How do you bisect to a layer? What fix matches each cause class?

28.17 When to Retry — Refined

Theory: Retry only transient, non-deterministic failures with evidence of external instability (TimeoutError, network blip — never assertion failure). Confirm pass-on-retry history and `testInfo.retry` pattern (fails attempt 1, passes attempt 2 = transient signature). Configure limited retries: 2 on CI (0 locally to expose bugs fast), exponential backoff, cleanup between attempts. Log retry statistics; still file flake fix (retry is detector, not cure). Use retries as classifier: `passed` (first-try) / `flaky` (pass-on-retry) / `failed` (all fail) — while addressing async waits, resources, isolation debt. Enterprise Relevance: Retry policy separates signal from noise without hiding either. Flaky bucket (pass-on-retry) gets tickets + SLA; failed bucket blocks; passed bucket ships.

```
export default defineConfig({ retries: process.env.CI ? 2 : 0 });
test('t', async ({ page }, testInfo) => {
  if (testInfo.retry) await cleanCaches(); // fresh state per attempt
});
```

High-Stakes Scenario: Retries mask a real race (passes 2nd try 90%) for 3 months — race ships to prod as double-charge. Triage: flaky-bucket review weekly (every entry needs root-cause ticket), retry-success rate tracked (rising = degrading), fix SLA by flake age. Anti-Patterns: Retries without classification (all green looks same); retries locally (hides bugs during dev); infinite retries; no cleanup between attempts (retry inherits corrupted state). Qs: What evidence justifies a retry? How do retries classify failures?

28.18 When NOT to Retry — Refined

Theory: Do not retry deterministic failures — retries hide bugs and waste resources. Fail fast on: consistent assertion errors, schema mismatches, broken selectors, 4xx logic errors. Check: all retries fail identically, or history shows 100% failure (deterministic signature). Quarantine with `test.fail`, `xfail`, or skip instead of inflating retries. Fix product code, selector, seed, or browser support directly. Reserve retries for proven infra flakes — never for real regressions. Enterprise Relevance: Retrying deterministic failures is the most expensive anti-pattern in CI: 3 retries × 2min × 100 failures = 10h burned daily hiding bugs that fail identically every time.

```
// deterministic → fail fast, no retry value:
test('price math', async ({ page }) => {
  // fails identically 10/10 → product bug, not flake → ticket, no retry
  await expect(page.getByTestId('total')).toHaveText('$100.00');
});
test.fail(true, 'known bug BUG-123', async () => { /* expected-fail tracking */ });
```


High-Stakes Scenario: Broken selector retried 3x per run × 50 runs = 150 wasted executions before anyone looks (all identical failures). Triage: identical-failure detector (same error 3x → auto-quarantine + ticket, stop retrying), `test.fail` for known bugs (tracks fix without noise). Anti-Patterns: Retrying assertion failures; retries as "stability" (hides regressions); no identical-failure detection; `xfail` without ticket (forgotten). Qs: Deterministic vs transient signature? What replaces retries for known bugs?